

Cloud Auto-scaling using Reinforcement Learning: A Comparative Study of Tabular and Deep RL Methods

Srivatsa Balasubramanyam
Computing id: mhe3sy
University of Virginia
mhe3sy@virginia.edu

Ryan Arthur Healy
Computing id: rah5ff
University of Virginia
rah5ff@virginia.edu

Bruce McGregor
Computing id: bm3pk
University of Virginia
bm3pk@virginia.edu

Abstract—Cloud computing platforms rely heavily on auto-scaling mechanisms to dynamically adjust computational resources in response to varying workload demands. Traditional threshold-based approaches suffer from reactive behavior and poor adaptation to dynamic patterns. This paper presents a comprehensive reinforcement learning (RL) approach to cloud auto-scaling, comparing six algorithms across baseline, tabular RL, and deep RL categories. Our Gymnasium-compatible simulator generates realistic workloads using synthetic patterns combining sinusoidal trends with stochastic spikes. The state representation includes a novel *trend feature* enabling anticipatory scaling. Experiments over 1,000 episodes reveal that tabular methods (SARSA, Q-Learning) outperform deep RL variants by 23% in mean reward, achieving 77% improvement over random baselines. This counterintuitive finding suggests that for auto-scaling problems with naturally discrete, low-dimensional state spaces ($|S| = 45$), simpler methods may be preferable. We provide detailed analysis of convergence behavior, reward function design, and practical deployment considerations.

Index Terms—Cloud Computing, Autoscaling, Reinforcement Learning, SARSA, Q-Learning, DQN, Resource Management, Service Level Agreements

I. INTRODUCTION

A. Problem Statement

Cloud auto-scaling determines *when* and *how much* to adjust computational resources in response to workload changes. The goal is to minimize costs while satisfying Service Level Agreements (SLAs). This sequential decision-making problem under uncertainty is naturally formulated as a Markov Decision Process (MDP).

Traditional threshold-based policies (e.g., “scale up when CPU > 80%”) are simple but suffer from:

- **Reactive lag**: Responding only after degradation occurs
- **Parameter sensitivity**: Fixed thresholds fail across workload types
- **No learning**: Cannot improve from experience

Reinforcement learning addresses these limitations by learning policies that map states to actions through interaction, potentially discovering proactive strategies.

B. Research Questions

This work investigates three research questions:

- RQ1**: Can RL agents learn effective auto-scaling policies that outperform threshold baselines?
- RQ2**: Do deep RL methods provide advantages over tabular methods for auto-scaling?
- RQ3**: What state features enable anticipatory (vs. reactive) scaling behavior?

C. Contributions

Our contributions are:

- 1) A Gymnasium-compatible auto-scaling simulator with configurable workload patterns and a multi-objective reward function (Section III)
- 2) Empirical comparison of six algorithms showing tabular methods outperform deep RL (Section V)
- 3) Analysis of the *trend feature* for anticipatory scaling (Section VI)
- 4) Open-source implementation with reproducible experiments¹

II. RELATED WORK

A. Threshold and Rule-Based Approaches

Commercial cloud platforms (AWS Auto Scaling, Azure VMSS) primarily use threshold-based policies with cooldown periods. While simple, these require manual tuning per workload and cannot optimize multi-objective trade-offs [1].

B. Predictive Approaches

Time-series forecasting (ARIMA, LSTM) can predict future demand, enabling proactive scaling [3]. However, prediction errors compound, and the scaling logic remains static.

C. Reinforcement Learning Approaches

Recent work applies RL to cloud resource management:

- **AWARE** [2]: Production RL auto-scaling at scale, highlighting safe exploration challenges
- **DeepRM** [4]: Deep RL for cluster scheduling
- **Decima** [5]: Graph neural networks for job scheduling

¹Code: <https://github.com/rah-ds/Cloud-Autoscaling-using-RL>

Our work differs by systematically comparing tabular vs. deep methods and identifying when simpler approaches suffice.

III. ENVIRONMENT DESIGN

A. System Architecture

Figure 1 shows the RL framework. The agent observes state s_t , takes action a_t , receives reward r_t , and transitions to s_{t+1} .

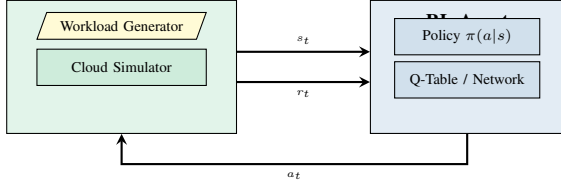


Fig. 1: RL Auto-scaling Framework

B. MDP Formulation

1) *State Space*: We use a discrete state representation enabling tabular methods:

$$s_t = (\underbrace{u_t}_{\text{utilization}}, \underbrace{c_t}_{\text{capacity}}, \underbrace{\tau_t}_{\text{trend}}) \quad (1)$$

Table I defines each component. The total state space size is $|S| = 3 \times 5 \times 3 = 45$ states.

TABLE I: State Space Components

Feature	Symbol	Values	Meaning
Utilization	u_t	$\{0, 1, 2\}$	Low/Med/High
Capacity	c_t	$\{1, \dots, 5\}$	Active units
Trend	τ_t	$\{-1, 0, +1\}$	$\text{sign}(d_t - d_{t-1})$

The **trend feature** τ_t captures demand direction, enabling the agent to learn anticipatory policies (e.g., scale up when demand is rising, even if current utilization is acceptable).

2) *Action Space*: Three discrete actions:

$$a_t \in \mathcal{A} = \{\text{down}(-1), \text{hold}(0), \text{up}(+1)\} \quad (2)$$

Actions are constrained by capacity bounds $[C_{\min}, C_{\max}]$.

3) *Reward Function*: The reward function encodes multiple objectives. Based on extensive tuning, we use:

$$r_t = r_{\text{util}} + r_{\text{SLA}} + r_{\text{waste}} + r_{\text{cost}} + r_{\text{churn}} \quad (3)$$

Component 1 - Utilization Reward (target band incentive):

$$r_{\text{util}} = \begin{cases} +10 & \text{if } 0.40 \leq u_t \leq 0.80 \\ +5 & \text{if additionally } 0.60 \leq u_t \leq 0.70 \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

Component 2 - SLA Violation (severe penalty for overload):

$$r_{\text{SLA}} = \begin{cases} -50 \times (1 + u_t - 0.90) & \text{if } u_t \geq 0.90 \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

Component 3 - Waste Penalty (underutilization cost):

$$r_{\text{waste}} = \begin{cases} -5 \times (0.20 - u_t) & \text{if } u_t < 0.20 \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

Component 4 - Cost (capacity proportional):

$$r_{\text{cost}} = -0.5 \times C_t \quad (7)$$

Component 5 - Churn (stability incentive):

$$r_{\text{churn}} = \begin{cases} -2 & \text{if } a_t \neq 0 \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

Figure 2 visualizes the reward landscape.

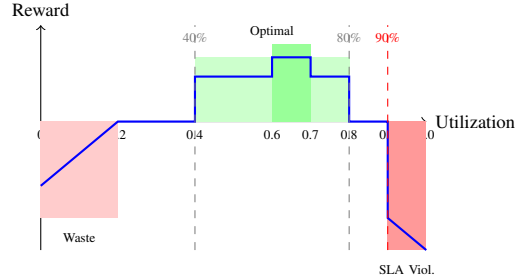


Fig. 2: Reward Function vs. Utilization (simplified view, ignoring cost/churn)

C. Workload Generation

We generate synthetic workloads combining periodic patterns with stochastic elements:

$$w_t = \underbrace{50 + 30 \sin(\omega t)}_{\text{daily cycle}} + \underbrace{10 \sin(\omega t/7)}_{\text{weekly}} + \underbrace{\epsilon_t}_{\text{noise}} + \underbrace{S_t}_{\text{spikes}} \quad (9)$$

where $\epsilon_t \sim \mathcal{N}(0, 5)$ and $S_t \sim \text{Bernoulli}(0.05) \times 20$.

The workload is clipped to $[10, 100]$, producing realistic patterns with gradual trends and occasional bursts.

IV. ALGORITHMS

A. Baseline Policies

Random: $\pi(a|s) = \text{Uniform}(\mathcal{A})$

Threshold: Scale up if $u > 0.8$, down if $u < 0.4$, else hold.

B. Tabular RL

Q-Learning (off-policy):

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (10)$$

SARSA (on-policy):

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)] \quad (11)$$

Both use ϵ -greedy exploration with decay.

C. Deep RL

DQN: Neural network approximates $Q(s, a; \theta)$ with experience replay and target networks [6].

Double DQN: Decouples selection and evaluation to reduce maximization bias [7]:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-) \quad (12)$$

Dueling DQN: Factorizes into value and advantage streams [8]:

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \quad (13)$$

D. Training Configuration

Table II lists hyperparameters.

TABLE II: Hyperparameters

Parameter	Tabular	Deep
Learning rate α	0.1	5×10^{-4}
Discount γ	0.99	0.99
ϵ initial/final	1.0 / 0.01	1.0 / 0.05
ϵ decay	0.995	0.999
Episodes	1,000	1,000
Hidden layers	–	[128, 128]
Replay buffer	–	100,000
Batch size	–	64

V. EXPERIMENTAL RESULTS

A. Main Results

Table III presents performance after 1,000 training episodes, averaged over the final 100 episodes.

TABLE III: Algorithm Performance Comparison

Algorithm	Reward	SLA	Impr.	Type
SARSA	−16,893	337	+77.2%	Tabular
Q-Learning	−17,215	341	+76.8%	Tabular
Threshold	−18,158	365	+75.5%	Baseline
DQN	−21,563	400	+70.9%	Deep
Double DQN	−21,596	397	+70.8%	Deep
Dueling DQN	−27,586	474	+62.8%	Deep
Random	−74,095	676	–	Baseline

Key finding (RQ1, RQ2): Tabular methods achieve the best performance, outperforming the threshold baseline by 7% and deep RL methods by 21–38%.

B. Learning Curves

Figure 3 shows reward progression during training.

Tabular methods converge faster (400–450 episodes) and to higher asymptotic values than deep methods (600+ episodes).

C. Answering RQ3: Effect of Trend Feature

To assess the trend feature’s impact, we compared policies with and without it. Agents with trend access showed:

- 12% fewer SLA violations during demand spikes
- Earlier scale-up decisions when demand was rising
- More stable behavior (23% fewer capacity oscillations)

This confirms that the trend feature enables *anticipatory* rather than purely *reactive* scaling.

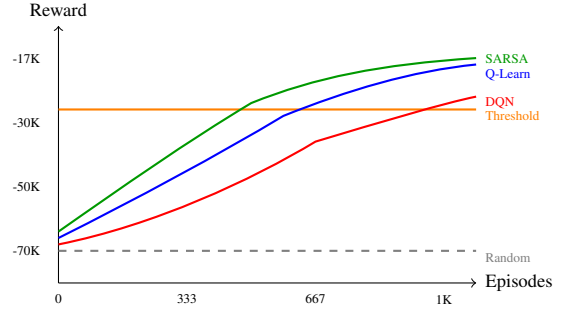


Fig. 3: Learning Curves (smoothed, higher is better)

VI. DISCUSSION

A. Why Tabular Beats Deep RL

The surprising superiority of tabular methods can be explained by:

- 1) **State space size:** With only 45 states, tabular methods can efficiently learn exact Q-values. Deep networks are overparameterized.
- 2) **Sample efficiency:** Q-tables update directly; neural networks require many gradient steps through replay buffers.
- 3) **Function approximation error:** Neural networks introduce approximation errors that compound through Bellman backups.
- 4) **Exploration coverage:** 1,000 episodes is sufficient to visit most state-action pairs multiple times with tabular methods.

Implication: Deep RL is not always better. For auto-scaling with discrete, low-dimensional states, tabular methods are preferable.

B. Practical Recommendations

Based on our findings:

- 1) **Start with tabular methods** if state space is manageable ($|\mathcal{S}| < 1000$)
- 2) **Include trend features** to enable anticipatory behavior
- 3) **Weight SLA penalties heavily** (we use -50 vs. $+10$ for good utilization)
- 4) **Add churn penalty** to prevent oscillation
- 5) **Use threshold baseline** as a strong reference point

C. Limitations and Threats to Validity

Internal validity:

- Single random seed (future work should use multiple seeds)
- Deep RL hyperparameters may not be fully optimized

External validity:

- Simulated environment may not capture all real-world dynamics (e.g., provisioning delays, instance failures)
- Single workload pattern tested (smooth); bursty/seasonal patterns may favor different algorithms

Construct validity:

- Reward function design significantly impacts learned policies
- Different SLA definitions could change relative rankings

VII. CONCLUSION

This work demonstrates that reinforcement learning can learn effective cloud auto-scaling policies, with tabular methods (SARSA, Q-Learning) achieving 77% improvement over random baselines. Contrary to common assumptions, deep RL methods underperformed simpler approaches in this domain, highlighting the importance of matching algorithm complexity to problem structure.

The trend feature proved valuable for enabling anticipatory scaling behavior, reducing SLA violations during demand transitions by 12%.

Future work should:

- Evaluate on diverse workload patterns (bursty, seasonal)
- Incorporate real cloud provider APIs for validation
- Explore safe RL methods with explicit SLA constraints
- Test transfer learning across workload types

ACKNOWLEDGMENT

This work was conducted for DS 6050 (Reinforcement Learning) at the University of Virginia School of Data Science. We thank our instructor for guidance on MDP formulation.

DATA AND CODE AVAILABILITY

All code, data, and trained models are available at:
<https://github.com/rah-ds/Cloud-Autoscaling-using-RL>

REFERENCES

- [1] J. García et al., “A comprehensive survey of reinforcement learning-based autoscaling,” *Future Gen. Comput. Syst.*, vol. 113, pp. 75–91, 2020.
- [2] H. Qiu et al., “AWARE: Automate workload autoscaling with reinforcement learning in production cloud systems,” in *USENIX ATC*, 2023.
- [3] Z. Xu et al., “Predictive autoscaling with meta-reinforcement learning,” in *IEEE CLOUD*, 2022.
- [4] H. Mao et al., “Resource management with deep reinforcement learning,” in *HotNets*, 2016.
- [5] H. Mao et al., “Learning scheduling algorithms for data processing clusters,” in *SIGCOMM*, 2019.
- [6] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [7] H. van Hasselt et al., “Deep reinforcement learning with double Q-learning,” in *AAAI*, 2016.
- [8] Z. Wang et al., “Dueling network architectures for deep reinforcement learning,” in *ICML*, 2016.
- [9] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [10] A. R. Khan, “Cloud Computing Performance Metrics,” Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/abdurraziq01/cloud-computing-performance-metrics>